

AI OPERATIONS PORTFOLIO · USER MANUAL

ops-profiler

A warehouse-design analytics engine that turns raw operational data into a sourced, decision-ready design package.

John White

AI Operations & Optimization

[linkedin.com/in/johnwhiteimse](https://www.linkedin.com/in/johnwhiteimse) · github.com/johnwhite-ai/ai-operations-portfolio

Contents

- 01** Executive Summary
- 02** The Problem
- 03** How It Works
- 04** The Differentiators
- 05** What One Command Produces
- 06** A Worked Example
- 07** Design Rationale & Tradeoffs
- 08** What This Project Demonstrates

01 Executive Summary

ops-profiler is a warehouse-design analytics engine. Given a customer's raw warehouse-management-system exports — a year of order movement, inventory snapshots, and physical master data — it produces a complete, fully sourced design package: demand percentiles, a pick-type work profile, storage requirements, first-principles system sizing (pick-station and bot counts), and a headcount ROI model that justifies the capital purchase.

It compressed roughly a week of senior-analyst work per engagement into a single command, produced identical methodology every run, and kept every value traceable to its source — defensible enough to price a multi-million-dollar system on.

THE ONE-LINE VERSION

Most data tools stop at "here are your numbers." ops-profiler keeps going — through system sizing and into the financial case — and proves every figure. That is the difference between a data tool and a decision tool.

02 The Problem

When a robotics company evaluates a warehouse for automation, a solutions engineer must answer hard quantitative questions from whatever data the customer can provide: How many orders per day, at peak? What does the picking work actually look like? How many storage positions, pick stations, and bots will the system need? And the question that closes the deal — how much labor cost does this displace?

Done by hand in spreadsheets, this had three failure modes:

- **Slow.** Roughly a week of senior-analyst work per engagement.
- **Inconsistent.** Two engineers profiled the same data differently, producing different design points.
- **Provenance evaporated.** Three weeks later, nobody could say whether a number was confirmed by the customer or assumed by the team. In a design review, that distinction decides whether the proposal survives scrutiny.

The underlying pattern is universal to operations: the answer is sitting in the data, but there is no automated, trustworthy path from the data to the decision.

03 How It Works

A four-stage pipeline takes raw files all the way to a sourced design package. It runs at any data richness level — a full year of WMS exports at one end, a handful of summary metrics at the other — and produces design-usable output either way.



The four-stage pipeline: ingest, profile, design, ROI/output. AI is confined to the extract edge; everything else is deterministic.

Stage 1 — Ingest

Raw exports are normalized and de-duplicated, and every record is stamped with provenance: which file, which sheet, which cell. Nothing enters the system without a source attached.

Stage 2 — Profile

The engine computes the demand signal (order, line, and unit percentiles, plus seasonality), the pick-type profile (piece vs. case vs. pallet, which determines the automation approach), and the storage profile (pallets per SKU, positions needed, velocity pareto).

Stage 3 — Design

From the profiled demand, the engine derives first-principles system sizing — pick-station and bot counts — using throughput rates, cycle times, utilization, and explicitly stated assumptions. The math is shown, never hidden in a template.

Stage 4 — ROI / Output

The engine builds the headcount ROI model (current vs. post-automation FTEs and the resulting labor savings) and assembles the customer-facing walkthroughs and the engineering brief. Every value carries its confidence tag and source.

04 The Differentiators

Four design decisions separate this from a typical analytics script. Each was a deliberate choice, and each is what makes the output trustworthy enough to act on.

DIFFERENTIATOR 1 — IT REACHES A FINANCIAL DECISION

The pipeline does not stop at a profile. It keeps going through first-principles system sizing and into a headcount ROI model. It produced the savings figure that justified the system purchase, with every input sourced. That end-to-end reach — raw data to dollars — is rare.

DIFFERENTIATOR 2 — FIRST-PRINCIPLES SIZING, MATH EXPOSED

Station and bot counts are derived from line volume, cycle times, utilization, and stated assumptions — never from a lookup table. A reviewer can see exactly how a "6 stations" recommendation was reached and challenge any single input. That is what made it survive design-review scrutiny.

DIFFERENTIATOR 3 — CONFIDENCE FROM PROVENANCE

Every value is tagged Confirmed, Estimated, or Missing — and the tag is assigned by one rule set based on WHERE the value came from, not what it is. A value the customer stated is Confirmed; a derived or AI-extracted value is Estimated; a required value never found is Missing. One rule, applied identically every run.

DIFFERENTIATOR 4 — DETERMINISTIC CORE, AI AT THE EDGES

Parsing, percentile math, and sizing are deterministic Python. The AI is used only for the two genuinely fuzzy jobs — mapping inconsistent column headers and reading values out of prose — and its output is always checked against the canonical schema before acceptance. The AI proposes; the rules dispose. It can never invent a parameter or touch the math.

AI PROPOSES · RULES DISPOSE

the AI maps messy headers and reads prose — it never touches the math



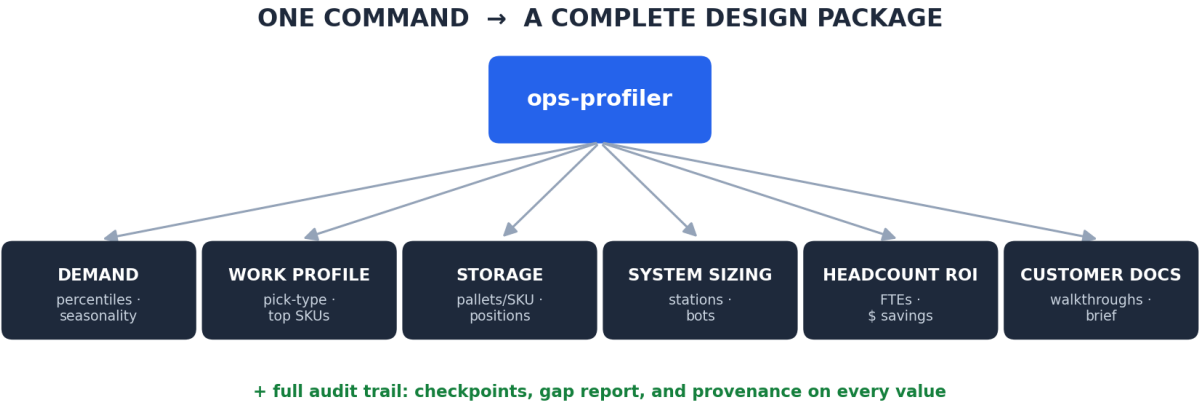
A hallucinated field is rejected before it can enter the design package.

The checked AI edge: a suggested mapping is validated against the schema before it can enter the design package. A hallucinated field is rejected.

05

What One Command Produces

For a single engagement, from one run, the engine generates a complete design package:



One command fans out into a complete, sourced design package plus a full audit trail.

Output	What it answers
Demand analysis	Order/line/unit percentiles (avg, P95, P99, peak) and seasonality
Pick-type profile	Piece vs. case vs. pallet split — decides the automation approach
Storage profile	Pallets per SKU, positions needed, velocity pareto
System sizing	First-principles pick-station and bot counts, math shown
Headcount ROI	Current vs. post-automation FTEs and annual labor savings
Customer + engineering docs	Alignment walkthrough, proposal walkthrough, structural brief
Audit trail	Checkpoints at every stage, a gap report, full provenance

06 A Worked Example

The sample below is a real generated output, anonymized. It shows the first-principles sizing method with the math fully exposed — the thing that made the output defensible in a design review.

Pick-station count estimate (illustrative)

```

Design lines/day (P95)           = 1,640           (from order percentiles)
Productive station-hours/day     = 8.5             (85% utilization assumption)
Lines per station per day        = 120 x 8.5 = 1,020
Stations needed (P95)           = 1,640 / 1,020 = 1.61 -> 2 (steady state)

Peak-day stress (P99 = 2,910):
  2,910 / 1,020 = 2.85 -> 3 stations

With redundancy + operational buffer -> 6 stations recommended

```

Every input is labeled and sourced; the single assumption (85% utilization) is stated explicitly. A reviewer who disagrees with the utilization figure can change one number and see the recommendation move. Nothing is a black box.

WHY THIS MATTERS

An engineer can walk into a design review and, for any number on the slide, instantly say where it came from and how confident the team is in it. That is the difference between a proposal that survives scrutiny and one that does not.

07

Design Rationale & Tradeoffs

A few decisions were not obvious, and the reasoning behind them is part of what the tool demonstrates.

Why deterministic core instead of an end-to-end LLM?

An LLM asked to read a spreadsheet and "return the design" is fast to prototype and impossible to trust — it will confidently produce a station count with no traceable basis. Confining the AI to header-mapping and prose-extraction, behind a schema check, keeps the speed where AI helps while keeping the numbers auditable. The cost is more engineering up front; the payoff is output you can defend to a skeptical customer.

Why surface gaps instead of filling them?

It is tempting to have the system guess at missing inputs so the output looks complete. That is exactly wrong for a design review. A visible Missing tag, surfaced up front as the next question to ask the customer, is more valuable than a confident wrong number discovered live in front of the client.

Why scale across data richness?

Customers rarely arrive with clean, complete data. Building a tool that only works with a full WMS export would mean it sits idle on most engagements. Designing it to degrade gracefully — full pipeline when the data is rich, sparse-input path when it is not — is what made it usable in the real world.

08

What This Project Demonstrates

Mapped explicitly to the capabilities an AI Operations or Optimization role screens for:

Capability	How this project proves it
Operations-research analytics	Percentile design points, first-principles system sizing, throughput modeling
Business-impact modeling	A full headcount ROI model that justified a capital purchase
Trustworthy AI system design	Deterministic core; AI confined to checked edges; never touches the math
Data engineering	Ingest, normalize, and de-duplicate a year of multi-file WMS exports
Provenance architecture	Every value traceable; confidence assigned by one rule set
Operational impact	A week of analyst work compressed to one command, output defensible enough to price on

THE TAKEAWAY

This is the same pattern applied across all operations work: take raw, messy data and build the system that turns it into a decision someone can act on — and can trust.